COMP2054-ADE Lec 11

The Vector Data Type, and "Amortised Analysis"

Andrew Parkes

Resizable arrays

- These are simply arrays, that automatically – hidden to the user - grow in size when try to add new elements at the end of the array, and that would otherwise overrun the end
  - Usually, can also shrink them afterwards, but that will not be relevant here
- Basic idea is simply to create a new array of a larger size, and copy across the old elements
  - Hide this implementation in an ADT
  - Often called a “Vector” ADT

Vector ADT – basic get/set

- https://docs.oracle.com/javase/8/docs/api/java/util/Vector.html
  - But there are equivalents in many languages
- Can be used like an array, call it V, but wrapping it with get/set methods:
  - get(int k) – gets the element at index k – basically like V[k]
  - set(int k, Element e) equivalent of V[k] = e
- Also, have the usual size()
- So valid indices are 0,…,size()-1

Vector ADT – “add” / “push”

- Also have an operation to add an element after the “nominal end” of the array

  add(Element e)

- Adds a new element but just after the usual range of 0,…,size()-1,
  - i.e. acts like a “push” when treating the Vector as a stack
- When needed, the underlying array will increase in size to cope
  - rather than throwing an exception

Vector ADT – CDT

- Underneath the ADT we will assume it is implemented as a simple array
  - For simplicity, we will do a Vector of int, i.e. just consider the stored Element's are simply int
- Core of the class:

```
class Vector {
    int[] A;
    int size; // number of elements stored
    int get(int k) { return A[k];}
    void set(int k, int e) { A[k]=e;}
    …
}
```

Vector: size vs. capacity

- When we construct a Vector object we need to also create, 'new', the array A
- We need to distinguish:
  - "Capacity" The length of the array A
  - "Size" : The total number of elements stored in A – often will write as 'n'
- These need not be equal: some indices might be unused. E.g. Capacity=7 size=5

0	1	2	3	4	5	6
78	43	1	4	2	-	-

Vector: add – easy cases

- If size < capacity, then we already have space to store a new element. Just place at the correct position and increment size.

```
add(Element e) {
    if ( size < capacity ) }
          A[size] = e
          size++
    } ...
```

Note this is simply O(1), as usual for "set" in an array

add(25)

0	1	2	3	4	5	6
78	43	1	4	2	25	-

add(4)

0	1	2	3	4	5	6
78	43	1	4	2	25	4

Vector: add – hard case

- If size = capacity, the array is full, and we first create a new array with a new larger capacity

```
resize(int newCap) {
        newA = new int[newCap]
        foreach k in 0,…,size-1
            newA[k] = A[k]
        A = newA     //  and the old A will be garbage collected
}
```

Cost is O(size) due to the copying, (etc.)

E.g. if pick newCap = capacity + 1

0	1	2	3	4	5	6
78	43	1	4	2	25	4

Gives

0	1	2	3	4	5	6	7
78	43	1	4	2	25	4	-

Then can do the usual add to place element at A[7]

Vector: add – costs

- The plain add is simply O(1)
- Any resize is O(size) as need to copy all the array
- So, we have two cases:

if (size < capacity)	cost O(1)
if (size = capacity)	cost O(size) for resize and then cost O(1) for the add

Vector: picking newCap

- We can pick any newCap > capacity
- Will consider two policies

- "**Incremental**" - newCap = capacity + d
  where d >= 1 is some constant

- "**Doubling**" - newCap = 2 * capacity

We will do some analysis of how well they perform:

Analysis of "add" Cost

- If we simply take the worst-case, then the cost of add is O(n) due to a potential resize
  - (Writing, using 'n' for the size)
- This would be independent of resize policy so such an analysis would not be useful

- However, generally, we do not always resize
  - Might do 'many add' operations, but 'few resizes'
  - How, can the analysis account for this !?

Analysis of "add" Cost (2)

- How do account for 'many add' operations, but 'few resizes' ?
- In practice, we often do a long sequence of 'add' operations
  - E.g. if reading data from a file, without knowing how many data elements we need
- So, we can
  1. consider the total cost of a (long) sequence of add operations
  2. compute the average cost per 'add' in that sequence

Analysis of "add" Cost (3)

For simplicity we consider the cost of starting with an empty Vector and then doing N 'add' operations

We then compute/measure

T(N) = total cost of the entire sequence of N 'add' operations

the **mean cost per operation over the complete sequence will then be T(N)/N**

We call this the "**amortised cost**" of the operation (Spelling: UK: "amortise" US: "amortize")

“Amortise” = “Spread out”

- See https://dictionary.cambridge.org/dictionary/english/amortize

  or similar if you are not familiar with the word.
- It refers to paying off debts but “spread out” over a period of time.
- Similar, to the way a mortgage for a house is paid back over many years, as opposed to needing to pay all in one go.
- Think of “amortised analysis” in the same way: paying the “resize costs” with a delay and “spread out” over many operations

Incremental Analysis: Example

Take d = 3, starting from empty, and capacity=3 and count the costs of a sequence of add(0).
(For simplicity, we just use some arbitrary scale for the costs; this does not affect the final Big-Oh answers.)

1. [- - -] size=0, cap=3, no resize: cost = 1 gives
2. [0 - -] size=1, cap=3, no resize: cost = 1 gives
3. [0 0 -] size=2, cap=3, no resize: cost = 1 gives
4. [0 0 0] size=3, cap=3, RESIZE cost = 3
   'add' cost = 3 + 1 gives
5. [0 0 0 0 - -] size=4, cap=6, no resize; cost = 1
6. [0 0 0 0 0 -] size=5, cap=6, no resize; cost = 1
7. [0 0 0 0 0 0] size=6, cap=6, RESIZE cost = 6
   'add' cost = 6 + 1 gives
8. [0 0 0 0 0 0 - -] size= 7, cap=9, no resize; cost = 1

Etc.

Incremental Analysis: Example

Take d = 3, starting from empty, and capacity=3 and count the costs of a sequence of add(0)
Sequence of costs:

1,1,1,3+1,1,1,6+1,1,1,9+1,1,1,12+1,1,1,15+1,1,1

Exercise (offline): Explicitly compute the sums to find T(N) and so T(N)/N as a function of N.
Plot graphs – and do an analysis.

See the appropriate lab session !

Incremental Analysis:

- Every 'add' costs the usual '1':
  - This just contributes N*1 to T(N)
- If n = "size so far":
  - every d operations we incur a resize cost of n
- Sizes at a resize will be a sequence

  d, 2d, 3d, …
- Giving overall cost of resizes:

  $d + 2d + 3d + \ldots + N = d\,(1 + 2 + 3 + \ldots + N/d\,)$
  - (Number of resizes needed is just N/d – up to irrelevant "off-by-one" issues)
- The arithmetic sum is O(N^2).
- Hence, T(N) is O(N^2)
- Hence, amortised cost is O(N) or O(size)

Doubling Analysis: Example

Starting from empty, and capacity=2 and count the costs of a sequence of add(0).

1. [0 -] size=1, cap=2, no resize: cost = 1 gives
2. [0 0] size=2, cap=2
   RESIZE cost = 2 'add' cost = 2 + 1 gives
3. [0 0 0 -] size=3, cap=4, no resize: cost = 1 gives
4. [0 0 0 0] size=4, cap=4
   RESIZE cost = 4 'add' cost = 4 + 1 gives
5. …
6. [0 0 0 0 0 0 0 0] size=8, cap=8
   RESIZE cost = 8 'add' cost = 8 + 1 gives
7. …

Etc.

Doubling Analysis: Costs

We get a sequence of costs:

1,
2+1,1,
4+1,1,1,1,
8+1,1,1,1,1,1,1,1,
16+1,1,1,1,1,…

Amortise = "Buy now, pay later"
We can split the resize cost over the following resize-free steps in each group "until the next resize".

1,
1+1,**1**+1, // from split of the '2' into a sequence of "1 +"
1+1,**1**+1,**1**+1,**1**+1, // from split of the '4'
1+1,**1**+1,**1**+1,**1**+1,**1**+1,**1**+1,**1**+1,**1**+1, // from split of the '8'
1+1,**1**+1, …

In this view the amortised cost is "1+1", so is O(1) per "add"

Doubling Analysis: Costs (2)

We get a sequence of costs:

1,
2+1,1,
4+1,1,1,1,
8+1,1,1,1,1,1,1,1,
16+1,1,1,1,1,…

The resize costs themselves are a geometric series

$1 + 2 + 4 + 8 + 16 + \ldots + 2^k$

And these will stop when the last term is (at worst) N

If $N = 2^k$ then $k = \log_2(N)$

Which matches the usual result for the number of iterations when "halving or doubling each time"

Recall: Geometric sums

Want to find S

$S = 1 + 2 + 2^2 + 2^3 + \ldots + 2^{\boldsymbol{k}}$

Standard trick:

$2S = \quad 2 + 2^2 + 2^3 + \ldots + 2^k + 2^{k+1}$

So

$2S - S = 2^{k+1} - 1$

Hence $S = 2^{k+1} - 1$

I.e. "the next term minus one"

Doubling Analysis: Costs (2)

We get a sequence of costs:

1,
2+1,1,
4+1,1,1,1,
8+1,1,1,1,1,1,1,1,
16+1,1,1,1,1,…

The resize costs themselves are a geometric series

$$1 + 2 + 4 + 8 + 16 + \ldots + 2^k$$

These would add up to "size at the next doubling – 1" and since we had $2^k = N$ we would get $2^{(k+1)} = 2N$

Hence, the total cost of the resizes is O(N)

The amortised cost of the resize is "O(N)/N" i.e. O(1)

Amortised Cost of "Add"

- Incremental strategy: capacity += const.
  - Worst case: O(N) per 'add'
  - T(N) is O(N^2)
  - Amortised cost (per add) is O(N)
    - This is bad, as many of the 'add' are just O(1)
  - Even after amortising, the resizes cost O(N)
- Doubling strategy: capacity *= 2
  - Worst case: O(N) per 'add'
  - T(N) is O(N)
  - Amortised cost (per add) is O(1)
    - This is good, as a plain 'add' is also just O(1)
  - The Big-Oh hides actual extra cost of the resize operations – as this amortises to a constant

General Amortised Analysis

In general:

- Some operation might take worst case time, T_W
- If do a sequence of N operations, then:
  - Call the actual cost T(N)
  - T(N) is clearly O(N * T_W)
  - But in some cases, the actual cost can be strictly smaller i.e. we can have that
    T(N) is o(N * T_W) [little-oh]
  - Then the "average over the sequence" is T(N)/N is the more useful as it tells us a better bound on the cost of long sequences of the operation

General Amortised Analysis (2)

[[Not assessed !!]]

- In some data structures the amortised cost can be challenging to compute.
- E.g. “Union Find” is a data structure used to manipulate (disjoint) sets, e.g. se
  - https://en.wikipedia.org/wiki/Disjoint-set_data_structure

- In good implementations, the operations can have amortised complexity involving the “Inverse Ackermann” function
  - which is not O(1) but grows very slowly,
  - e.g. it is o(log n), and o(log log n) and …
- The amortised analysis shows that the operations are “near constant”

C/C++ comments

- Doubling vector size might be made even more efficient if use “realloc” rather than “malloc” or “new”
  - Internally it can (often, but not always) just extend the space allocated to the array, and so avoid the need for a copy
- Also, could use a “memcpy” which is direct copy rather than via individuals in a for loop

Question (pause and try):

- Why is amortised analysis different from the average case analysis?

Question:

- Why is amortised analysis different from the average case analysis?
- Answer:
  - "Amortised": (long) **real sequence** of dependent operations, and gains from high costs not tending to be closely spaced
  - vs.
  - "Average": **Set** of operations and that in general are independent and might all be very costly

Offline Exercises/Discussion

- When might you still want to use incremental increase rather than doubling?

- Why is it doubling rather than tripling? Or increasing by some other constant factor?

  - Try redoing the analysis with a arbitrary growth factor *b*
  - *As ever: implement it all!*
  - *Try it in the lab session !*

Expectations

- The “Vector” data structure
- The big-Oh costs of various single operations
- The amortised cost of a sequence of operations
- The amortised complexities of different strategies for resizing of the underlying array